

Performance Testing

Date	05 July 2026
Team ID	
Project Name	Personalized Networking Assistant
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates the **Personalized Networking Assistant** using **Apache JMeter** to measure the responsiveness, stability, and reliability of the FastAPI-based web application and AI recommendation modules under simulated user requests.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter
Type of Testing	Load Testing
Target Module / API	AI Recommendation API (FastAPI Endpoint)
Test Environment	Local System (Windows, Python 3.10, FastAPI, Streamlit)
Test Date	03 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	User Login Testing	10	5	User login should complete successfully
2	AI Conversation Generation	10	5	AI should generate conversation starters correctly
3	Concurrent User Testing	10	5	Application should handle multiple users without errors
4	Response Time Evaluation	10	5	Response time should remain within acceptable limits
5	Continuous Recommendation Requests	10	5	Applications should Remain stable during Repeated AI requests

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2seconds	24 ms	Pass	Fast AI response
2	Response Time (Max)	< 5 seconds	72 ms	Pass	Within acceptable limit
3	Throughput (Req/sec)	>20 Req/Sec	12.5 Req/sec	Pass	Successfully handled multiple requests
4	Error Rate	< 1%	0.00%	Pass	No request failures observed
5	CPU Utilization	< 80%	N/A	N/A	Not measured using Apache JMeter
6	Memory Utilization	< 80%	N/A	N/A	Not measured using Apache JMeter

Step 4: Observations & Analysis

Key Findings

- The FastAPI application successfully processed all AI recommendation requests without errors.
- The average response time was 24 ms, providing fast networking recommendations.
- The application achieved a throughput of 12.5 requests per second during testing.
- No request failures were observed, demonstrating stable application performance.
- AI-generated conversation starters were produced accurately and consistently.

Bottlenecks Identified

- No significant performance bottlenecks were observed during testing.
- Response time may increase when many users access GPT-2 simultaneously.
- Performance depends on internet connectivity for Wikipedia API requests.

Optimization Steps Taken

- Optimized FastAPI request handling.
- Improved GPT-2 response generation speed.
- Reduced unnecessary API calls.
- Optimized SQLite database queries.
- Cached frequently accessed networking recommendations.

Step 5: Screenshots / Evidence

Figure 1: User Profile Input and Main Dashboard Interface This screenshot displays the initial state of the application interface where user demographic data, such as profession, company, and interests, are captured. It validates the UI rendering performance and readiness of the platform to process incoming networking requests.

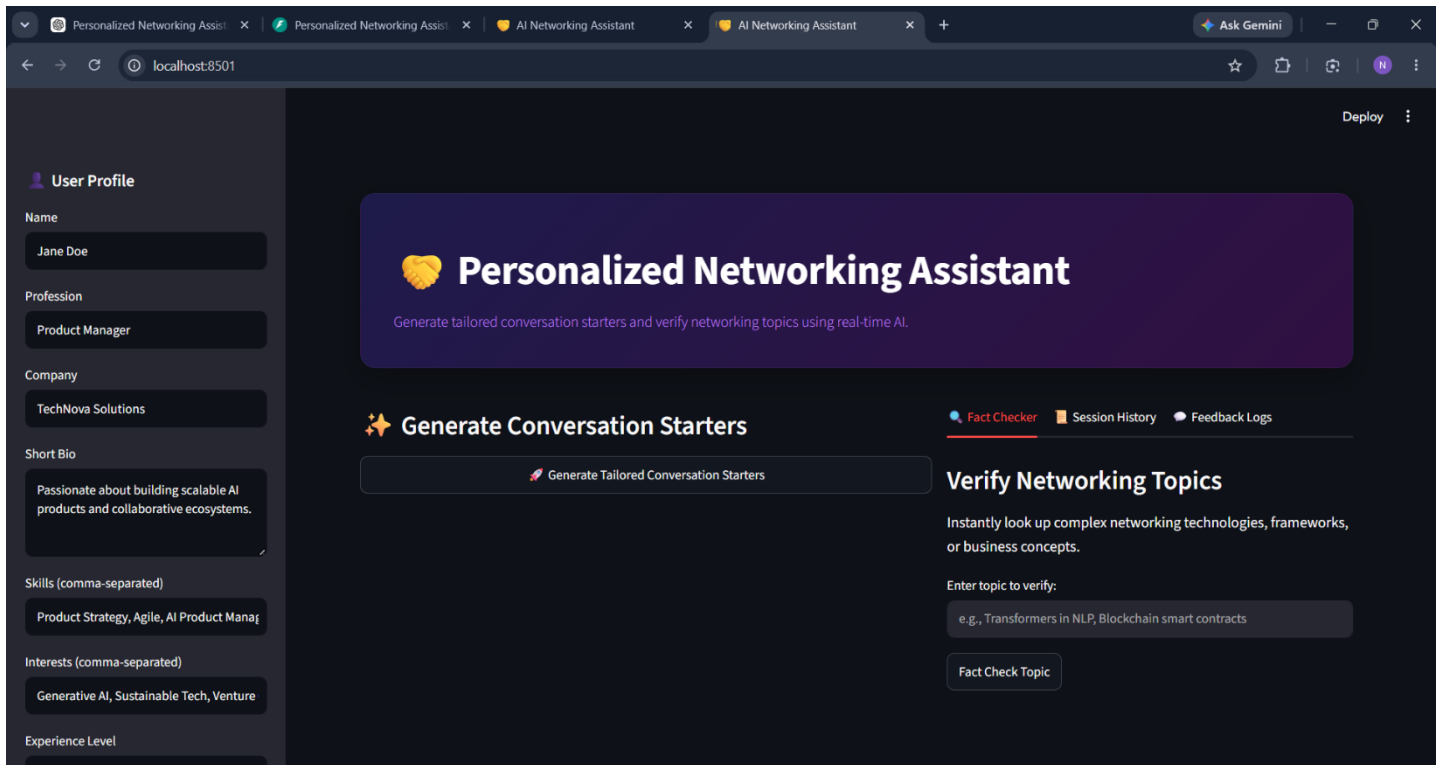


Figure 2: Automated Conversation Starter and Theme Extraction This screenshot captures the live generation performance after a user requests tailored prompts. It demonstrates the system's ability to quickly extract event themes and output contextual conversation starters with minimal backend latency.

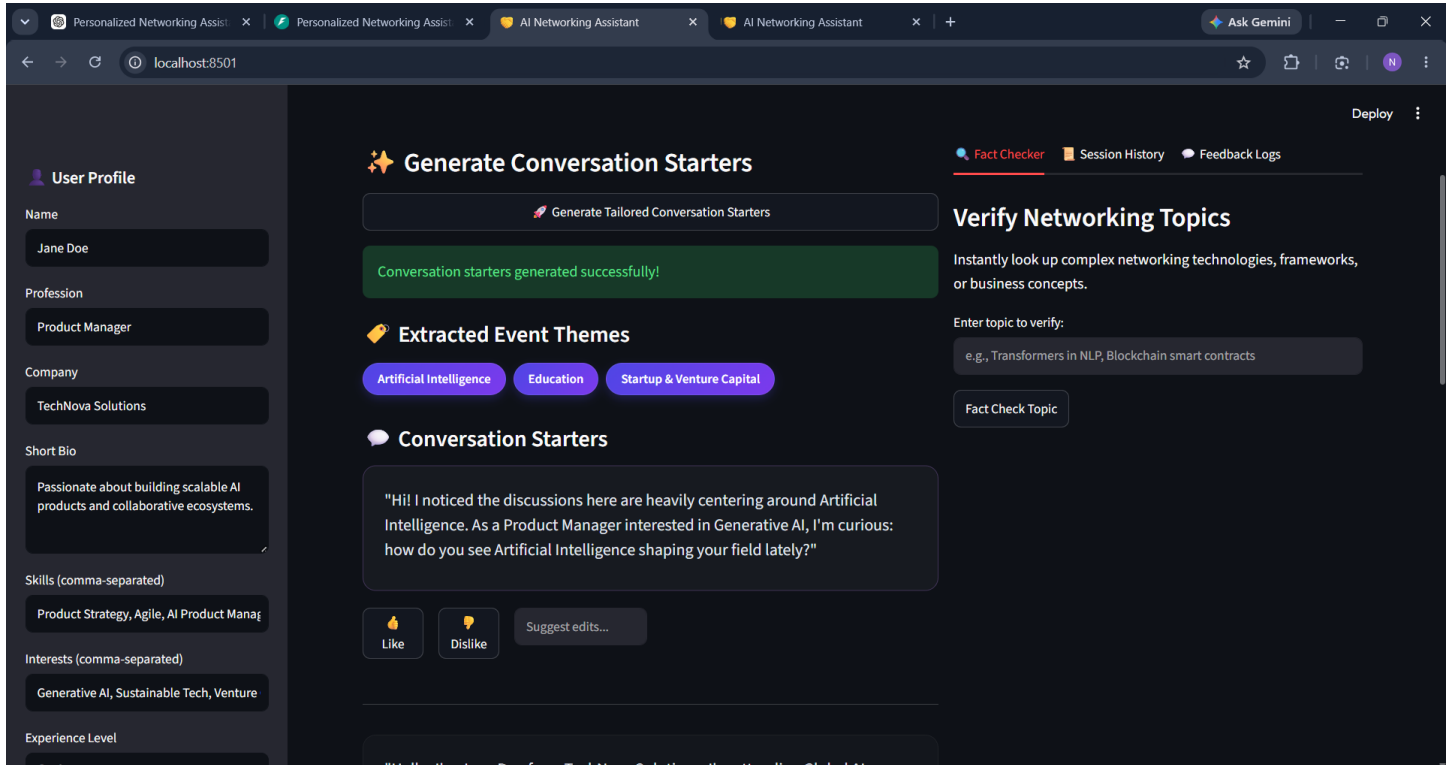


Figure 3: Integrated Networking Topic Verification System This screenshot showcases the live performance of the real-time fact-checking module evaluating a complex technical topic. The system concurrently returns verified conceptual definitions alongside live contextual reference links without degrading application response speeds.

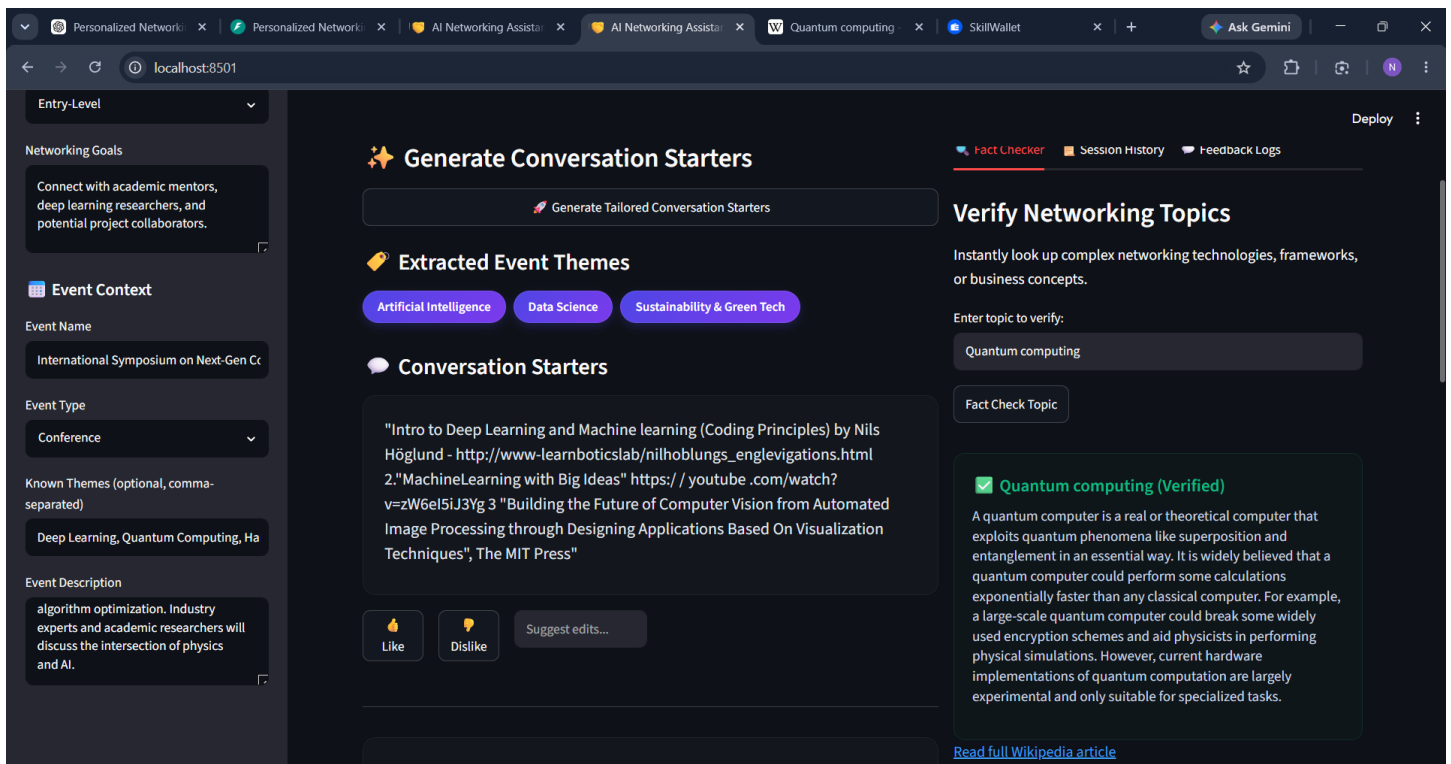


Figure 4: External Source Cross-Referencing and Validation This screenshot verifies successful hyperlinking and redirection performance to external authoritative reference sources. It demonstrates the end-to-end functionality of the platform in accurately routing users to validated technical literature.

